

1. Infrastructure as Code (IaC)

Infrastructure as Code is the practice of managing and creating infrastructure using code instead of manually configuring servers.

- Infrastructure can include servers, networks, databases, and cloud resources.
- Makes infrastructure repeatable and consistent.
- Common IaC tools include Terraform and AWS CloudFormation.

Example: Instead of manually creating an AWS EC2 instance, you define it in a Terraform configuration file.

2. Terraform

Terraform is an open-source IaC tool developed by HashiCorp.

It allows you to define and manage infrastructure using configuration files written in HCL (HashiCorp Configuration Language).

Terraform can manage:

- AWS/Azure/GCP resources
- Virtual machines
- Networks
- Databases
- DNS and other cloud services

3. Terraform State

Terraform State keeps track of the infrastructure that Terraform manages.

Usually stored in a file called: `terraform.tfstate`

It records information such as:

- Resources Terraform created
- Resource IDs
- Current configuration information
- Relationships between resources

State allows Terraform to compare the desired infrastructure with the actual infrastructure and determine what changes are required.

For teams, state can be stored remotely, for example in an Amazon S3 backend

4. Variables & Outputs

Variables allow you to make Terraform configurations reusable and customizable.

Example:

```
variable "instance_type" {  
    default = "t2.micro"  
}
```

5. Modules

A Terraform module is a reusable collection of Terraform configuration files.

Instead of writing the same infrastructure code repeatedly, you can create a module and reuse it.

For example:

```
modules/  
└─ web-server/  
    └─ main.tf  
    └─ variables.tf  
    └─ outputs.tf
```

Modules help with:

- Reusability
- Organization
- Consistency
- Managing large Terraform projects

6. Ansible

Ansible is an automation and configuration-management tool.

It is commonly used to:

- Install software
- Configure servers
- Create users
- Deploy applications
- Manage services

Ansible usually connects to Linux servers using SSH and executes tasks remotely.

Example:

```
Ansible → SSH → Server  
                ↓  
            Install Docker
```

7. Inventory

An Ansible Inventory contains the list of servers that Ansible manages.

Example:

```
[web_servers]  
server1 ansible_host=192.168.1.10  
server2 ansible_host=192.168.1.11
```

```
[db_servers]
database1 ansible_host=192.168.1.20
```

Servers can be organized into groups such as:

- Web servers
- Database servers
- Application servers

Ansible uses the inventory to know which machines to manage.

8. Playbooks

An Ansible Playbook is a YAML file containing instructions for configuring or managing servers.

Example:

```
- name: Install Docker
  hosts: web_servers
  become: true

tasks:
  - name: Install Docker
    apt:
      name: docker.io
      state: present
```

A playbook defines:

- What should be done
- Which servers it should be done on
- How it should be done

Playbooks make server configuration automated and repeatable.

9. Terraform vs Ansible

Terraform

Mainly used for infrastructure provisioning
Creates resources such as VMs, networks, databases
Uses HCL
Uses Terraform State
Declarative
Example: Create an EC2 instance

Ansible

Mainly used for configuration and automation
Configures software and operating systems
Uses YAML
Does not normally require a state file
Primarily task/playbook-based automation
Example: Install Docker on the EC2 instance